

School of Computer Science and Artificial Intelligence

Lab Assignment # 8.2

Program : B. Tech (CSE)

Specialization :AIML

Course Title : AI Assisted Coding

Course Code : 23CS002PC304

Semester : VI

Academic Session : 2025-2026

Name of Student : k.sathwika

Enrollment No. : 2303A52096

Batch No. : 33

Date :10/02/26

TASK_01:

Prompt:

Generate pytest test cases for a Python function `is_even(n)` handling integers, zero, negatives, and invalid inputs. Use TDD.

Code:

Test-Driven Development (TDD) for `is_even(n)`

Following the TDD methodology, we will first write the tests for the `is_even` function, then create a minimal implementation of the function, and finally run the tests to see them fail. After observing the failures, we will update the `is_even` function to pass all tests.

Test Scenarios:

- Positive Integers: Both even and odd.
- Zero: Zero is considered an even number.
- Negative Integers: Both even and odd.
- Invalid Inputs: Non-integer types (e.g., float, string, None).

```
import pytest

# Define the test cases
def test_is_even_positive_even():
    assert is_even(2) is True
    assert is_even(4) is True
    assert is_even(100) is True

def test_is_even_positive_odd():
    assert is_even(1) is False
    assert is_even(3) is False
    assert is_even(99) is False

def test_is_even_zero():
    assert is_even(0) is True

def test_is_even_negative_even():
    assert is_even(-2) is True
    assert is_even(-4) is True
    assert is_even(-100) is True

def test_is_even_negative_odd():
    assert is_even(-1) is False
    assert is_even(-3) is False
    assert is_even(-99) is False

def test_is_even_invalid_input_float():
    with pytest.raises(TypeError):
        is_even(2.5)
    with pytest.raises(TypeError):
        is_even(-0.0)

def test_is_even_invalid_input_string():
    with pytest.raises(TypeError):
        is_even("hello")
    with pytest.raises(TypeError):
        is_even("2")

def test_is_even_invalid_input_none():
    with pytest.raises(TypeError):
        is_even(None)

def test_is_even_invalid_input_list():
    with pytest.raises(TypeError):
        is_even([2])
```

Step 2: Implement a Minimal `is_even` Function

This initial implementation is intentionally incomplete to demonstrate how tests will fail in a TDD workflow. You will expand on this after running the tests.

```
def is_even(n):  
    # This is a minimal, placeholder implementation.  
    # It will make some tests fail, demonstrating TDD.  
    if not isinstance(n, int):  
        raise TypeError("Input must be an integer")  
    return n % 2 == 0
```

Step 3: Install Pytest

Run the following cell to install pytest if you haven't already.

```
%pip install pytest  
  
Requirement already satisfied: pytest in /usr/local/lib/python3.12/dist-packages (8.4.2)  
Requirement already satisfied: iniconfig>=1 in /usr/local/lib/python3.12/dist-packages (from pytest) (2.3.0)  
Requirement already satisfied: packaging>=20 in /usr/local/lib/python3.12/dist-packages (from pytest) (26.0)  
Requirement already satisfied: pluggy<2,>=1.5 in /usr/local/lib/python3.12/dist-packages (from pytest) (1.6.0)  
Requirement already satisfied: pygments>=2.7.2 in /usr/local/lib/python3.12/dist-packages (from pytest) (2.19.2)
```

Step 4: Run the Tests

Now, let's run the tests. You should see some failures, which is expected at this stage of TDD.

```
import pytest
import os
import tempfile
import shutil # Import shutil for robust directory removal
import uuid # Import uuid for unique filenames

# Content of the 'is_even' function from cell 31106c60
is_even_func_code = """
def is_even(n):
    # This is a minimal, placeholder implementation.
    # It will make some tests fail, demonstrating TDD.
    if not isinstance(n, int):
        raise TypeError("Input must be an integer")
    return n % 2 == 0
"""

# Content of the pytest test cases from cell 5900ca3f
test_cases_code = """
import pytest

# Define the test cases
def test_is_even_positive_even():
    assert is_even(2) is True
    assert is_even(4) is True
    assert is_even(100) is True

def test_is_even_positive_odd():
    assert is_even(1) is False
    assert is_even(3) is False
    assert is_even(99) is False

def test_is_even_zero():
    assert is_even(0) is True

def test_is_even_negative_even():
    assert is_even(-2) is True
    assert is_even(-4) is True
    assert is_even(-100) is True

def test_is_even_negative_odd():
    assert is_even(-1) is False
    assert is_even(-3) is False
    assert is_even(-99) is False

def test_is_even_invalid_input_float():
    with pytest.raises(TypeError):
        is_even(2.5)
    with pytest.raises(TypeError):
        is_even(-0.0)

def test_is_even_invalid_input_string():
    with pytest.raises(TypeError):
        is_even("hello")

```

```

        is_even("hello")
        with pytest.raises(TypeError):
            is_even("2")

def test_is_even_invalid_input_none():
    with pytest.raises(TypeError):
        is_even(None)

def test_is_even_invalid_input_list():
    with pytest.raises(TypeError):
        is_even([2])
"""

# Combine the function and test code into a single string
full_test_script = is_even_func_code + test_cases_code

# Create a temporary directory and file
temp_dir = tempfile.mkdtemp()
# Generate a unique filename using uuid to prevent import conflicts
unique_filename_suffix = uuid.uuid4().hex
temp_test_filename = f"test_is_even_generated_{unique_filename_suffix}.py"
temp_test_file_path = os.path.join(temp_dir, temp_test_filename)

# Write the combined script to the temporary file
with open(temp_test_file_path, "w") as f:
    f.write(full_test_script)

# Store current working directory and change to temp_dir
original_cwd = os.getcwd()
os.chdir(temp_dir)

try:
    # Run pytest on the temporary file using its filename (since we are in the directory)
    # We use '-v' for verbose output to see which tests are run and their outcomes
    pytest.main([temp_test_filename, '-v'])
finally:
    # Change back to the original working directory
    os.chdir(original_cwd)
    # Clean up the temporary file and directory using shutil.rmtree
    if os.path.exists(temp_dir):
        shutil.rmtree(temp_dir)

```

Output:

```

... Requirement already satisfied: pytest in /usr/local/lib/python3.12/dist-packages (8.4.2)
Requirement already satisfied: iniconfig>=1 in /usr/local/lib/python3.12/dist-packages (from pytest) (2.3.0)
Requirement already satisfied: packaging>=20 in /usr/local/lib/python3.12/dist-packages (from pytest) (26.0)
Requirement already satisfied: pluggy<2,>=1.5 in /usr/local/lib/python3.12/dist-packages (from pytest) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in /usr/local/lib/python3.12/dist-packages (from pytest) (2.19.2)

```

```
*** test session starts ***
platform linux -- Python 3.12.12, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
rootdir: /tmp/tmp96umysr5
plugins: langsmith-0.6.8, typeguard-4.4.4, anyio-4.12.1
collecting ... collected 9 items

test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_positive_even PASSED [ 11%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_positive_odd PASSED [ 22%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_zero PASSED [ 33%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_negative_even PASSED [ 44%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_negative_odd PASSED [ 55%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_invalid_input_float PASSED [ 66%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_invalid_input_string PASSED [ 77%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_invalid_input_none PASSED [ 88%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_invalid_input_list PASSED [100%]

===== 9 passed in 0.06s =====
```

TASK_02:

Prompt:

Generate pytest test cases for `to_uppercase(text)` and `to_lowercase(text)` handling empty strings, mixed case, and invalid inputs using TDD.

Code:

Step 1: Write Pytest Test Cases for Case Conversion Functions

Here are the pytest test cases for `to_uppercase` and `to_lowercase`, covering various scenarios.

```
import pytest

# Test cases for to_uppercase
def test_to_uppercase_normal():
    assert to_uppercase("hello") == "HELLO"

def test_to_uppercase_mixed_case():
    assert to_uppercase("Hello World") == "HELLO WORLD"

def test_to_uppercase_already_uppercase():
    assert to_uppercase("PYTHON") == "PYTHON"

def test_to_uppercase_empty_string():
    assert to_uppercase("") == ""

def test_to_uppercase_with_numbers_symbols():
    assert to_uppercase("Hello 123!") == "HELLO 123!"

def test_to_uppercase_invalid_input_none():
    with pytest.raises(TypeError):
        to_uppercase(None)

def test_to_uppercase_invalid_input_int():
    with pytest.raises(TypeError):
        to_uppercase(123)

def test_to_uppercase_invalid_input_list():
    with pytest.raises(TypeError):
        to_uppercase(['a', 'b'])

# Test cases for to_lowercase
def test_to_lowercase_normal():
    assert to_lowercase("WORLD") == "world"

def test_to_lowercase_mixed_case():
    assert to_lowercase("Hello World") == "hello world"

def test_to_lowercase_already_lowercase():
    assert to_lowercase("python") == "python"

def test_to_lowercase_empty_string():
    assert to_lowercase("") == ""

def test_to_lowercase_with_numbers_symbols():
    assert to_lowercase("HELLO 123!") == "hello 123!"

def test_to_lowercase_invalid_input_none():
    with pytest.raises(TypeError):
        to_lowercase(None)

def test_to_lowercase_invalid_input_int():
    with pytest.raises(TypeError):
        to_lowercase(123)

def test_to_lowercase_invalid_input_list():
    with pytest.raises(TypeError):
        to_lowercase(['A', 'B'])
```

Step 2: Implement Minimal Case Conversion Functions

These initial implementations are intentionally incomplete to demonstrate how tests will fail in a TDD workflow. You will expand on these after running the tests.

```
[14]
✓ On def to_uppercase(text):
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text.upper()

def to_lowercase(text):
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text.lower()
```


Step 3: Run the Tests

Now, let's run the tests. You should see some failures, which is expected at this stage of TDD.

```
[11] 1 import pytest
✓ On 2 import os
3 import tempfile
4 import shutil
5 import uuid

6
7 # Content of the 'to_uppercase' and 'to_lowercase' functions
8 string_func_code = """
9 def to_uppercase(text):
10     if not isinstance(text, str):
11         raise TypeError("Input must be a string")
12     return text
13
14 def to_lowercase(text):
15     if not isinstance(text, str):
16         raise TypeError("Input must be a string")
17     return text
18 """
19
20 # Content of the pytest test cases
21 test_cases_code = """
22 import pytest
23
24 # Test cases for to_uppercase
25 def test_to_uppercase_normal():
26     assert to_uppercase("hello") == "HELLO"
27
28 def test_to_uppercase_mixed_case():
29     assert to_uppercase("Hello World") == "HELLO WORLD"
30
31 def test_to_uppercase_already_uppercase():
32     assert to_uppercase("PYTHON") == "PYTHON"
33
34 def test_to_uppercase_empty_string():
35     assert to_uppercase("") == ""
36
37 def test_to_uppercase_with_numbers_symbols():
38     assert to_uppercase("Hello 123!") == "HELLO 123!"
39
40 def test_to_uppercase_invalid_input_none():
41     with pytest.raises(TypeError):
42         to_uppercase(None)
43
44 def test_to_uppercase_invalid_input_int():
45     with pytest.raises(TypeError):
46         to_uppercase(123)
47
48 def test_to_uppercase_invalid_input_list():
49     with pytest.raises(TypeError):
50         to_uppercase(['a', 'b'])
51
52 # Test cases for to_lowercase
53 def test_to_lowercase_normal():
54     assert to_lowercase("WORLD") == "world"
55
56 def test_to_lowercase_mixed_case():
```

```

    assert to_lowercase("WORLD") == "world"

def test_to_lowercase_mixed_case():
    assert to_lowercase("Hello World") == "hello world"

def test_to_lowercase_already_lowercase():
    assert to_lowercase("python") == "python"

def test_to_lowercase_empty_string():
    assert to_lowercase("") == ""

def test_to_lowercase_with_numbers_symbols():
    assert to_lowercase("HEllo 123!") == "hello 123!"

def test_to_lowercase_invalid_input_none():
    with pytest.raises(TypeError):
        to_lowercase(None)

def test_to_lowercase_invalid_input_int():
    with pytest.raises(TypeError):
        to_lowercase(123)

def test_to_lowercase_invalid_input_list():
    with pytest.raises(TypeError):
        to_lowercase(['A', 'B'])
...

# Combine the function and test code into a single string
full_test_script = string_func_code + test_cases_code

# Create a temporary directory and file
temp_dir = tempfile.mkdtemp()
unique_filename_suffix = uuid.uuid4().hex
temp_test_filename = f"test_string_case_generated_{unique_filename_suffix}.py"
temp_test_file_path = os.path.join(temp_dir, temp_test_filename)

# Write the combined script to the temporary file
with open(temp_test_file_path, "w") as f:
    f.write(full_test_script)

# Store current working directory and change to temp_dir
original_cwd = os.getcwd()
os.chdir(temp_dir)

try:
    # Run pytest on the temporary file using its filename (since we are in the directory)
    pytest.main([temp_test_filename, '-v'])
finally:
    # Change back to the original working directory
    os.chdir(original_cwd)
    # Clean up the temporary file and directory
    if os.path.exists(temp_dir):
        shutil.rmtree(temp_dir)

```

```

import pytest
import os
import tempfile
import shutil
import uuid

# Content of the 'to_uppercase' and 'to_lowercase' functions
string_func_code = """
def to_uppercase(text):
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text.upper()

def to_lowercase(text):
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text.lower()
"""

# Content of the pytest test cases from cell 17028486
test_cases_code = """
import pytest

# Test cases for to_uppercase
def test_to_uppercase_normal():
    assert to_uppercase("hello") == "HELLO"

def test_to_uppercase_mixed_case():
    assert to_uppercase("Hello World") == "HELLO WORLD"

def test_to_uppercase_already_uppercase():
    assert to_uppercase("PYTHON") == "PYTHON"

def test_to_uppercase_empty_string():
    assert to_uppercase("") == ""

def test_to_uppercase_with_numbers_symbols():
    assert to_uppercase("Hello 123!") == "HELLO 123!"

def test_to_uppercase_invalid_input_none():
    with pytest.raises(TypeError):
        to_uppercase(None)

def test_to_uppercase_invalid_input_int():
    with pytest.raises(TypeError):
        to_uppercase(123)

def test_to_uppercase_invalid_input_list():
    with pytest.raises(TypeError):
        to_uppercase(['a', 'b'])

# Test cases for to_lowercase
def test_to_lowercase_normal():
    assert to_lowercase("WORLD") == "world"

def test_to_lowercase_mixed_case():
    assert to_lowercase("Hello World") == "hello world"

def test_to_lowercase_already_lowercase():
    assert to_lowercase("python") == "python"

def test_to_lowercase_empty_string():
    assert to_lowercase("") == ""

```

```

    assert to_lowercase("") == ""

def test_to_lowercase_with_numbers_symbols():
    assert to_lowercase("HELLO 123!") == "hello 123!"

def test_to_lowercase_invalid_input_none():
    with pytest.raises(TypeError):
        to_lowercase(None)

def test_to_lowercase_invalid_input_int():
    with pytest.raises(TypeError):
        to_lowercase(123)

def test_to_lowercase_invalid_input_list():
    with pytest.raises(TypeError):
        to_lowercase(['A', 'B'])
'''

# Combine the function and test code into a single string
full_test_script = string_func_code + test_cases_code

# Create a temporary directory and file
temp_dir = tempfile.mkdtemp()
unique_filename_suffix = uuid.uuid4().hex
temp_test_filename = f"test_string_case_generated_{unique_filename_suffix}.py"
temp_test_file_path = os.path.join(temp_dir, temp_test_filename)

# Write the combined script to the temporary file
with open(temp_test_file_path, "w") as f:
    f.write(full_test_script)

# Store current working directory and change to temp_dir
original_cwd = os.getcwd()
os.chdir(temp_dir)

try:
    # Run pytest on the temporary file using its filename (since we are in the directory)
    pytest.main([temp_test_filename, '-v'])
finally:
    # Change back to the original working directory
    os.chdir(original_cwd)
    # Clean up the temporary file and directory
    if os.path.exists(temp_dir):
        shutil.rmtree(temp_dir)

```

Output:

```

*** ===== test session starts =====
platform linux -- Python 3.12.12, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /tmp/tmpn3eusejx
plugins: langsmith-0.6.8, typeguard-4.4.4, anyio-4.12.1
collecting ... collected 16 items

test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_uppercase_normal FAILED [ 6%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_uppercase_mixed_case FAILED [ 12%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_uppercase_already_uppercase PASSED [ 18%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_uppercase_empty_string PASSED [ 25%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_uppercase_with_numbers_symbols FAILED [ 31%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_uppercase_invalid_input_none PASSED [ 37%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_uppercase_invalid_input_int PASSED [ 43%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_uppercase_invalid_input_list PASSED [ 50%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_lowercase_normal FAILED [ 56%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_lowercase_mixed_case FAILED [ 62%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_lowercase_already_lowercase PASSED [ 68%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_lowercase_empty_string PASSED [ 75%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_lowercase_with_numbers_symbols FAILED [ 81%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_lowercase_invalid_input_none PASSED [ 87%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_lowercase_invalid_input_int PASSED [ 93%]
test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_lowercase_invalid_input_list PASSED [100%]

===== FAILURES =====
_____ test_to_uppercase_normal _____

def test_to_uppercase_normal():
> assert to_uppercase("hello") == "HELLO"
E       AssertionError: assert 'hello' == 'HELLO'
E
E       - HELLO
E       + hello

test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py:16: AssertionError
_____ test_to_uppercase_mixed_case _____

def test_to_uppercase_mixed_case():
> assert to_uppercase("Hello World") == "HELLO WORLD"
E       AssertionError: assert 'Hello World' == 'HELLO WORLD'
E
E       - HELLO WORLD
E       + Hello World

test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py:19: AssertionError
_____ test_to_uppercase_with_numbers_symbols _____

def test_to_uppercase_with_numbers_symbols():
> assert to_uppercase("Hello 123!") == "HELLO 123!"
E       AssertionError: assert 'Hello 123!' == 'HELLO 123!'
E
E       - HELLO 123!
E       + Hello 123!

```

```

test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py:28: AssertionError
_____ test_to_lowercase_normal _____

    def test_to_lowercase_normal():
>     assert to_lowercase("WORLD") == "world"
E     AssertionError: assert 'WORLD' == 'world'
E
E     - world
E     + WORLD

test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py:44: AssertionError
_____ test_to_lowercase_mixed_case _____

    def test_to_lowercase_mixed_case():
>     assert to_lowercase("Hello World") == "hello world"
E     AssertionError: assert 'Hello World' == 'hello world'
E
E     - hello world
E     ? ^         ^
E     + Hello World
E     ? ^         ^

test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py:47: AssertionError
_____ test_to_lowercase_with_numbers_symbols _____

    def test_to_lowercase_with_numbers_symbols():
>     assert to_lowercase("HELLO 123!") == "hello 123!"
E     AssertionError: assert 'HELLO 123!' == 'hello 123!'
E
E     - hello 123!
E     + HELLO 123!

test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py:56: AssertionError
===== short test summary info =====
FAILED test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_uppercase_normal
FAILED test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_uppercase_mixed_case
FAILED test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_uppercase_with_numbers_symbols
FAILED test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_lowercase_normal
FAILED test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_lowercase_mixed_case
FAILED test_string_case_generated_4a5be3cb0295489cb1c5f13cd5f0a44c.py::test_to_lowercase_with_numbers_symbols
===== 6 failed, 10 passed in 0.12s =====

```

```

*** ===== test session starts =====
platform linux -- Python 3.12.12, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /tmp/tmp96umysr5
plugins: langsmith-0.6.8, typeguard-4.4.4, anyio-4.12.1
collecting ... collected 9 items

test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_positive_even PASSED [ 11%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_positive_odd PASSED [ 22%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_zero PASSED [ 33%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_negative_even PASSED [ 44%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_negative_odd PASSED [ 55%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_invalid_input_float PASSED [ 66%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_invalid_input_string PASSED [ 77%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_invalid_input_none PASSED [ 88%]
test_is_even_generated_083423fd45e047289c14053b8f8741f2.py::test_is_even_invalid_input_list PASSED [100%]

===== 9 passed in 0.06s =====

```

```

===== test session starts =====
platform linux -- Python 3.12.12, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /tmp/tmpd9pggcki
plugins: langsmith-0.6.8, typeguard-4.4.4, anyio-4.12.1
collecting ... collected 16 items

test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_uppercase_normal PASSED [ 6%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_uppercase_mixed_case PASSED [ 12%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_uppercase_already_uppercase PASSED [ 18%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_uppercase_empty_string PASSED [ 25%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_uppercase_with_numbers_symbols PASSED [ 31%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_uppercase_invalid_input_none PASSED [ 37%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_uppercase_invalid_input_int PASSED [ 43%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_uppercase_invalid_input_list PASSED [ 50%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_lowercase_normal PASSED [ 56%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_lowercase_mixed_case PASSED [ 62%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_lowercase_already_lowercase PASSED [ 68%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_lowercase_empty_string PASSED [ 75%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_lowercase_with_numbers_symbols PASSED [ 81%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_lowercase_invalid_input_none PASSED [ 87%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_lowercase_invalid_input_int PASSED [ 93%]
test_string_case_generated_2d4face3e8ee4e9599e41057ca64d4d9.py::test_to_lowercase_invalid_input_list PASSED [100%]

===== 16 passed in 0.09s =====

```

TASK_03:

Prompt:

Generate pytest test cases for `sum_list(numbers)` handling empty lists, negative numbers, and non-numeric values using TDD.

Code:

```

+ Gemini
1 import pytest
import os
import tempfile
import shutil
import uuid

# Content of the 'sum_list' function (now updated)
sum_list_func_code = """
def sum_list(numbers):
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")
    total = 0
    for num in numbers:
        if not isinstance(num, (int, float)):
            raise TypeError("All elements in the list must be numbers")
        total += num
    return total
"""

# Content of the pytest test cases
test_cases_code = """
import pytest

# Test cases for sum_list
def test_sum_list_empty():
    assert sum_list([]) == 0

def test_sum_list_positive_integers():
    assert sum_list([1, 2, 3]) == 6

def test_sum_list_negative_integers():
    assert sum_list([-1, -2, -3]) == -6

def test_sum_list_mixed_positive_negative():
    assert sum_list([1, -2, 3]) == 2

def test_sum_list_zero():
    assert sum_list([0, 0, 0]) == 0

def test_sum_list_mixed_integers_floats():
    assert sum_list([1, 2.5, 3]) == 6.5

def test_sum_list_with_non_numeric_values():
    with pytest.raises(TypeError):
        sum_list([1, 'a', 3])
    with pytest.raises(TypeError):
        sum_list([1, None, 3])

def test_sum_list_invalid_input_not_list_int():
    with pytest.raises(TypeError):
        sum_list(10)

def test_sum_list_invalid_input_not_list_string():
    with pytest.raises(TypeError):
        sum_list("hello")
"""

# Combine the function and test code into a single string
full_test_script = sum_list_func_code + test_cases_code

# Create a temporary directory and file
temp_dir = tempfile.mkdtemp()
unique_filename_suffix = uuid.uuid4().hex
temp_test_filename = f"test_sum_list_generated_{unique_filename_suffix}.py"
temp_test_file_path = os.path.join(temp_dir, temp_test_filename)

# Write the combined script to the temporary file
with open(temp_test_file_path, "w") as f:
    f.write(full_test_script)

# Store current working directory and change to temp_dir
original_cwd = os.getcwd()
os.chdir(temp_dir)

try:
    # Run pytest on the temporary file using its filename (since we are in the directory)
    pytest.main([temp_test_filename, '-v'])
finally:
    # Change back to the original working directory
    os.chdir(original_cwd)
    # Clean up the temporary file and directory
    if os.path.exists(temp_dir):
        shutil.rmtree(temp_dir)

```

Output:


```
shutil.rmtree(temp_dir)

... ===== test session starts =====
platform linux -- Python 3.12.12, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /tmp/tmpFr_k3v9e
plugins: langsmith-0.6.8, typeguard-4.4.4, anyio-4.12.1
collecting ... collected 9 items

test_sum_list_generated_2341a2c2d9734d17b469f88c6c75b459.py::test_sum_list_empty PASSED [ 11%]
test_sum_list_generated_2341a2c2d9734d17b469f88c6c75b459.py::test_sum_list_positive_integers PASSED [ 22%]
test_sum_list_generated_2341a2c2d9734d17b469f88c6c75b459.py::test_sum_list_negative_integers PASSED [ 33%]
test_sum_list_generated_2341a2c2d9734d17b469f88c6c75b459.py::test_sum_list_mixed_positive_negative PASSED [ 44%]
test_sum_list_generated_2341a2c2d9734d17b469f88c6c75b459.py::test_sum_list_zero PASSED [ 55%]
test_sum_list_generated_2341a2c2d9734d17b469f88c6c75b459.py::test_sum_list_mixed_integers_floats PASSED [ 66%]
test_sum_list_generated_2341a2c2d9734d17b469f88c6c75b459.py::test_sum_list_with_non_numeric_values PASSED [ 77%]
test_sum_list_generated_2341a2c2d9734d17b469f88c6c75b459.py::test_sum_list_invalid_input_not_list_int PASSED [ 88%]
test_sum_list_generated_2341a2c2d9734d17b469f88c6c75b459.py::test_sum_list_invalid_input_not_list_string PASSED [100%]

===== 9 passed in 0.05s =====
```

TASK_04:

Prompt:

Generate pytest test cases for a StudentResult class with mark validation and pass/fail logic using TDD.

Code:

```
[20] import pytest
✓ Os

# Assume StudentResult class is defined elsewhere for testing purposes.
# For this step, we are only writing the tests, the class will be implemented later.
# Placeholder for StudentResult class for local testing and to avoid NameError
# when running this cell independently for syntax check. It will be replaced
# by the actual implementation in subsequent steps.
class StudentResult:
    def __init__(self, name, marks):
        if not isinstance(name, str) or name is None:
            raise TypeError("Name must be a string and not None.")
        if not name:
            raise ValueError("Name cannot be an empty string.")
        self._name = name
        self.marks = marks # Use setter for validation

    @property
    def name(self):
        return self._name

    @property
    def marks(self):
        return self._marks

    @marks.setter
    def marks(self, value):
        if not isinstance(value, (int, float)) or value is None:
            raise TypeError("Marks must be a numeric value and not None.")
        if not (0 <= value <= 100):
            raise ValueError("Marks must be between 0 and 100.")
        self._marks = value

    def is_passed(self):
        return self._marks >= 50

# Test cases for StudentResult class initialization

def test_student_result_init_valid_data():
    student = StudentResult("Alice", 85)
    assert student.name == "Alice"
    assert student.marks == 85

def test_student_result_init_valid_boundary_marks():
    student_min = StudentResult("Bob", 0)
    student_max = StudentResult("Charlie", 100)
    assert student_min.marks == 0
    assert student_max.marks == 100

def test_student_result_init_type_error_name_none():
    with pytest.raises(TypeError, match="Name must be a string and not None."):
        StudentResult(None, 70)

def test_student_result_init_type_error_name_non_string():
    with pytest.raises(TypeError, match="Name must be a string and not None."):
        StudentResult(123, 70)

def test_student_result_init_value_error_name_empty_string():
```

```
[20]
✓ Os
def test_student_result_init_value_error_name_empty_string():
    with pytest.raises(ValueError, match="Name cannot be an empty string."):
        StudentResult("", 70)

def test_student_result_init_type_error_marks_none():
    with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
        StudentResult("David", None)

def test_student_result_init_type_error_marks_non_numeric():
    with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
        StudentResult("Eve", "seventy")

def test_student_result_init_value_error_marks_too_low():
    with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
        StudentResult("Frank", -10)

def test_student_result_init_value_error_marks_too_high():
    with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
        StudentResult("Grace", 110)

# Test cases for marks setter validation

def test_student_result_set_marks_valid():
    student = StudentResult("Heidi", 70)
    student.marks = 90
    assert student.marks == 90

def test_student_result_set_marks_valid_boundary():
    student = StudentResult("Ivan", 70)
    student.marks = 0
    assert student.marks == 0
    student.marks = 100
    assert student.marks == 100

def test_student_result_set_marks_type_error_none():
    student = StudentResult("Judy", 70)
    with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
        student.marks = None
```

```
def test_student_result_set_marks_type_error_non_numeric():
    student = StudentResult("Karl", 70)
    with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
        student.marks = "eighty"

def test_student_result_set_marks_value_error_too_low():
    student = StudentResult("Liam", 70)
    with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
        student.marks = -5

def test_student_result_set_marks_value_error_too_high():
    student = StudentResult("Mia", 70)
    with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
        student.marks = 101

# Test cases for is_passed() method

def test_student_result_is_passed_above_threshold():
    student = StudentResult("Nora", 75) # 75 > 50
    assert student.is_passed() is True

def test_student_result_is_passed_at_threshold():
    student = StudentResult("Oscar", 50) # 50 >= 50
    assert student.is_passed() is True

def test_student_result_is_passed_below_threshold():
    student = StudentResult("Paul", 45) # 45 < 50
    assert student.is_passed() is False

def test_student_result_is_passed_zero_marks():
    student = StudentResult("Quinn", 0)
    assert student.is_passed() is False

def test_student_result_is_passed_full_marks():
    student = StudentResult("Ryan", 100)
    assert student.is_passed() is True
```

```
class StudentResult:
    def __init__(self, name, marks):
        self._name = name
        self._marks = marks

    @property
    def name(self):
        return self._name

    @property
    def marks(self):
        return self._marks

    @marks.setter
    def marks(self, value):
        # Intentionally no validation here to cause test failures
        self._marks = value

    def is_passed(self):
        # Intentionally incorrect logic to cause test failures
        return False

print("Minimal StudentResult class defined.")
```

Reasoning: The subtask is to run the previously defined pytest test cases against the minimal StudentResult class to demonstrate test failures in a TDD workflow. This requires dynamically creating a test file and executing pytest.

[22]
✓ 1s

```
import pytest
import os
import tempfile
import shutil
import uuid

# Content of the minimal StudentResult class
minimal_student_result_code = """
class StudentResult:
    def __init__(self, name, marks):
        self._name = name
        self._marks = marks

    @property
    def name(self):
        return self._name

    @property
    def marks(self):
        return self._marks

    @marks.setter
    def marks(self, value):
        # Intentionally no validation here to cause test failures
        self._marks = value

    def is_passed(self):
        # Intentionally incorrect logic to cause test failures
        return False
"""

# Content of the pytest test cases from cell c570ba3c
test_cases_code = """
import pytest

# Test cases for StudentResult class initialization

def test_student_result_init_valid_data():
    student = StudentResult("Alice", 85)
    assert student.name == "Alice"
    assert student.marks == 85

def test_student_result_init_valid_boundary_marks():
    student_min = StudentResult("Bob", 0)
    student_max = StudentResult("Charlie", 100)
    assert student_min.marks == 0
    assert student_max.marks == 100

def test_student_result_init_type_error_name_none():
    with pytest.raises(TypeError, match="Name must be a string and not None."):
        StudentResult(None, 70)

def test_student_result_init_type_error_name_non_string():
    with pytest.raises(TypeError, match="Name must be a string and not None."):
        StudentResult(123, 70)

```

```
[22]
✓ 1s def test_student_result_init_value_error_name_empty_string():
    with pytest.raises(ValueError, match="Name cannot be an empty string."):
        StudentResult("", 70)

def test_student_result_init_type_error_marks_none():
    with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
        StudentResult("David", None)

def test_student_result_init_type_error_marks_non_numeric():
    with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
        StudentResult("Eve", "seventy")

def test_student_result_init_value_error_marks_too_low():
    with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
        StudentResult("Frank", -10)

def test_student_result_init_value_error_marks_too_high():
    with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
        StudentResult("Grace", 110)

# Test cases for marks setter validation

def test_student_result_set_marks_valid():
    student = StudentResult("Heidi", 70)
    student.marks = 90
    assert student.marks == 90

def test_student_result_set_marks_valid_boundary():
    student = StudentResult("Ivan", 70)
    student.marks = 0
    assert student.marks == 0
    student.marks = 100
    assert student.marks == 100

def test_student_result_set_marks_type_error_none():
    student = StudentResult("Judy", 70)
    with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
        student.marks = None

def test_student_result_set_marks_type_error_non_numeric():
    student = StudentResult("Karl", 70)
    with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
        student.marks = "eighty"

def test_student_result_set_marks_value_error_too_low():
    student = StudentResult("Liam", 70)
    with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
        student.marks = -5

def test_student_result_set_marks_value_error_too_high():
    student = StudentResult("Mia", 70)
    with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
        student.marks = 101

# Test cases for is_passed() method

def test_student_result_is_passed_above_threshold():
    student = StudentResult("Mona", 75) # 75 > 50
```

```
Q Commands + Code + Text ▶ Run all

[22] ✓ 1s ▶ def test_student_result_is_passed_at_threshold():
    student = StudentResult("Oscar", 50) # 50 >= 50
    assert student.is_passed() is True

    def test_student_result_is_passed_below_threshold():
        student = StudentResult("Paul", 45) # 45 < 50
        assert student.is_passed() is False

    def test_student_result_is_passed_zero_marks():
        student = StudentResult("Quinn", 0)
        assert student.is_passed() is False

    def test_student_result_is_passed_full_marks():
        student = StudentResult("Ryan", 100)
        assert student.is_passed() is True
    """

    # Combine the function and test code into a single string
    full_test_script = minimal_student_result_code + test_cases_code

    # Create a temporary directory and file
    temp_dir = tempfile.mkdtemp()
    unique_filename_suffix = uuid.uuid4().hex
    temp_test_filename = f"test_student_result_generated_{unique_filename_suffix}.py"
    temp_test_file_path = os.path.join(temp_dir, temp_test_filename)

    # Write the combined script to the temporary file
    with open(temp_test_file_path, "w") as f:
        f.write(full_test_script)

    # Store current working directory and change to temp_dir
    original_cwd = os.getcwd()
    os.chdir(temp_dir)

    try:
        # Run pytest on the temporary file using its filename (since we are in the directory)
        # We use '-v' for verbose output to see which tests are run and their outcomes
        pytest.main([temp_test_filename, '-v'])
    finally:
        # Change back to the original working directory
        os.chdir(original_cwd)
        # Clean up the temporary file and directory using shutil.rmtree
        if os.path.exists(temp_dir):
            shutil.rmtree(temp_dir)
```

```
[23] ✓ 0s ▶ class StudentResult:
    def __init__(self, name, marks):
        # Name validation
        if not isinstance(name, str) or name is None:
            raise TypeError("Name must be a string and not None.")
        if not name:
            raise ValueError("Name cannot be an empty string.")
        self._name = name

        # Marks validation handled by the setter property
        self.marks = marks

    @property
    def name(self):
        return self._name

    @property
    def marks(self):
        return self._marks

    @marks.setter
    def marks(self, value):
        if not isinstance(value, (int, float)) or value is None:
            raise TypeError("Marks must be a numeric value and not None.")
        if not (0 <= value <= 100):
            raise ValueError("Marks must be between 0 and 100.")
        self._marks = value

    def is_passed(self):
        return self._marks >= 50

    print("Refined StudentResult class defined.")
```

```

import pytest
import os
import tempfile
import shutil
import uuid

# Content of the refined StudentResult class from cell 1ad75121
refined_student_result_code = """
class StudentResult:
    def __init__(self, name, marks):
        # Name validation
        if not isinstance(name, str) or name is None:
            raise TypeError("Name must be a string and not None.")
        if not name:
            raise ValueError("Name cannot be an empty string.")
        self._name = name

        # Marks validation handled by the setter property
        self.marks = marks

    @property
    def name(self):
        return self._name

    @property
    def marks(self):
        return self._marks

    @marks.setter
    def marks(self, value):
        if not isinstance(value, (int, float)) or value is None:
            raise TypeError("Marks must be a numeric value and not None.")
        if not (0 <= value <= 100):
            raise ValueError("Marks must be between 0 and 100.")
        self._marks = value

    def is_passed(self):
        return self._marks >= 50
"""

# Content of the pytest test cases from cell c570ba3c
test_cases_code = """
import pytest

# Test cases for StudentResult class initialization

def test_student_result_init_valid_data():
    student = StudentResult("Alice", 85)
    assert student.name == "Alice"
    assert student.marks == 85

def test_student_result_init_valid_boundary_marks():
    student_min = StudentResult("Bob", 0)
    student_max = StudentResult("Charlie", 100)
    assert student_min.marks == 0
    assert student_max.marks == 100

```

[24]
✓ Os

```
def test_student_result_init_type_error_name_none():
    with pytest.raises(TypeError, match="Name must be a string and not None."):
        StudentResult(None, 70)

def test_student_result_init_type_error_name_non_string():
    with pytest.raises(TypeError, match="Name must be a string and not None."):
        StudentResult(123, 70)

def test_student_result_init_value_error_name_empty_string():
    with pytest.raises(ValueError, match="Name cannot be an empty string."):
        StudentResult("", 70)

def test_student_result_init_type_error_marks_none():
    with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
        StudentResult("David", None)

def test_student_result_init_type_error_marks_non_numeric():
    with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
        StudentResult("Eve", "seventy")

def test_student_result_init_value_error_marks_too_low():
    with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
        StudentResult("Frank", -10)

def test_student_result_init_value_error_marks_too_high():
    with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
        StudentResult("Grace", 110)

# Test cases for marks setter validation

def test_student_result_set_marks_valid():
    student = StudentResult("Heidi", 70)
    student.marks = 90
    assert student.marks == 90

def test_student_result_set_marks_valid_boundary():
    student = StudentResult("Ivan", 70)
    student.marks = 0
    assert student.marks == 0
    student.marks = 100
    assert student.marks == 100

def test_student_result_set_marks_type_error_none():
    student = StudentResult("Judy", 70)
    with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
        student.marks = None

def test_student_result_set_marks_type_error_non_numeric():
    student = StudentResult("Karl", 70)
    with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
        student.marks = "eighty"

def test_student_result_set_marks_value_error_too_low():
    student = StudentResult("Liam", 70)
    with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
        student.marks = -5
```

```
assert student.is_passed() is True

def test_student_result_is_passed_at_threshold():
    student = StudentResult("Oscar", 50) # 50 >= 50
    assert student.is_passed() is True

def test_student_result_is_passed_below_threshold():
    student = StudentResult("Paul", 45) # 45 < 50
    assert student.is_passed() is False

def test_student_result_is_passed_zero_marks():
    student = StudentResult("Quinn", 0)
    assert student.is_passed() is False

def test_student_result_is_passed_full_marks():
    student = StudentResult("Ryan", 100)
    assert student.is_passed() is True
"""

# Combine the function and test code into a single string
full_test_script = refined_student_result_code + test_cases_code

# Create a temporary directory and file
temp_dir = tempfile.mkdtemp()
unique_filename_suffix = uuid.uuid4().hex
temp_test_filename = f"test_student_result_generated_{unique_filename_suffix}.py"
temp_test_file_path = os.path.join(temp_dir, temp_test_filename)

# Write the combined script to the temporary file
with open(temp_test_file_path, "w") as f:
    f.write(full_test_script)

# Store current working directory and change to temp_dir
original_cwd = os.getcwd()
os.chdir(temp_dir)

try:
    # Run pytest on the temporary file using its filename (since we are in the directory)
    # We use '-v' for verbose output to see which tests are run and their outcomes
    pytest.main([temp_test_filename, '-v'])
finally:
    # Change back to the original working directory
    os.chdir(original_cwd)
    # Clean up the temporary file and directory using shutil.rmtree
    if os.path.exists(temp_dir):
        shutil.rmtree(temp_dir)
```


Output:

```
*** Minimal StudentResult class defined.
```

```
*** ===== test session starts =====
platform linux -- Python 3.12.12, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /tmp/tmpv12gw7sb
plugins: langsmith-0.6.8, typeguard-4.4.4, anyio-4.12.1
collecting ... collected 20 items

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_valid_data PASSED [ 5%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_valid_boundary_marks PASSED [ 10%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_type_error_name_none FAILED [ 15%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_type_error_name_non_string FAILED [ 20%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_value_error_name_empty_string FAILED [ 25%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_type_error_marks_none FAILED [ 30%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_type_error_marks_non_numeric FAILED [ 35%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_value_error_marks_too_low FAILED [ 40%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_value_error_marks_too_high FAILED [ 45%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_set_marks_valid PASSED [ 50%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_set_marks_valid boundary PASSED [ 55%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_set_marks_type_error_name FAILED [ 60%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_set_marks_type_error_non_numeric FAILED [ 65%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_set_marks_value_error_too_low FAILED [ 70%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_set_marks_value_error_too_high FAILED [ 75%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_is_passed_above_threshold FAILED [ 80%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_is_passed_at_threshold FAILED [ 85%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_is_passed_below_threshold PASSED [ 90%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_is_passed_zero_marks PASSED [ 95%]
test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_is_passed_full_marks FAILED [100%]

===== FAILURES =====
_____ test_student_result_init_type_error_name_none _____

    def test_student_result_init_type_error_name_none():
>     with pytest.raises(TypeError, match="Name must be a string and not None."):
E       Failed: DID NOT RAISE <class 'TypeError'>

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:40: Failed
_____ test_student_result_init_type_error_name_non_string _____

    def test_student_result_init_type_error_name_non_string():
>     with pytest.raises(TypeError, match="Name must be a string and not None."):
E       Failed: DID NOT RAISE <class 'TypeError'>

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:44: Failed
_____ test_student_result_init_value_error_name_empty_string _____

    def test_student_result_init_value_error_name_empty_string():
>     with pytest.raises(ValueError, match="Name cannot be an empty string."):
E       Failed: DID NOT RAISE <class 'ValueError'>

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:48: Failed
_____ test_student_result_init_type_error_marks_none _____

    def test_student_result_init_type_error_marks_none():
>     with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
E       Failed: DID NOT RAISE <class 'TypeError'>
```

```

E      Failed: DID NOT RAISE <class 'TypeError'>

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:52: Failed
_____ test_student_result_init_type_error_marks_non_numeric _____

    def test_student_result_init_type_error_marks_non_numeric():
>       with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
           ~~~~~
E      Failed: DID NOT RAISE <class 'TypeError'>

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:56: Failed
_____ test_student_result_init_value_error_marks_too_low _____

    def test_student_result_init_value_error_marks_too_low():
>       with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
           ~~~~~
E      Failed: DID NOT RAISE <class 'ValueError'>

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:60: Failed
_____ test_student_result_init_value_error_marks_too_high _____

    def test_student_result_init_value_error_marks_too_high():
>       with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
           ~~~~~
E      Failed: DID NOT RAISE <class 'ValueError'>

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:64: Failed
_____ test_student_result_set_marks_type_error_none _____

    def test_student_result_set_marks_type_error_none():
        student = StudentResult("Judy", 70)
>       with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
           ~~~~~
E      Failed: DID NOT RAISE <class 'TypeError'>

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:83: Failed
_____ test_student_result_set_marks_type_error_non_numeric _____

    def test_student_result_set_marks_type_error_non_numeric():
        student = StudentResult("Karl", 70)
>       with pytest.raises(TypeError, match="Marks must be a numeric value and not None."):
           ~~~~~
E      Failed: DID NOT RAISE <class 'TypeError'>

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:88: Failed
_____ test_student_result_set_marks_value_error_too_low _____

    def test_student_result_set_marks_value_error_too_low():
        student = StudentResult("Liam", 70)
>       with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
           ~~~~~
E      Failed: DID NOT RAISE <class 'ValueError'>

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:93: Failed
_____ test_student_result_set_marks_value_error_too_high _____

    def test_student_result_set_marks_value_error_too_high():
        student = StudentResult("Mia", 70)

```

```
[22]  shutil.rmtree(temp_dir)
✓ 15

... E      Failed: DID NOT RAISE <class 'ValueError'>

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:93: Failed
_____ test_student_result_set_marks_value_error_too_high _____

    def test_student_result_set_marks_value_error_too_high():
        student = StudentResult("Mia", 70)
    >       with pytest.raises(ValueError, match="Marks must be between 0 and 100."):
    ~~~~~
E      Failed: DID NOT RAISE <class 'ValueError'>

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:98: Failed
_____ test_student_result_is_passed_above_threshold _____

    def test_student_result_is_passed_above_threshold():
        student = StudentResult("Mora", 75) # 75 > 50
    >       assert student.is_passed() is True
E      assert False is True
E      + where False = is_passed()
E      +   where is_passed = <test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.StudentResult object at 0x7b011577a960>.is_passed

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:105: AssertionError
_____ test_student_result_is_passed_at_threshold _____

    def test_student_result_is_passed_at_threshold():
        student = StudentResult("Oscar", 50) # 50 >= 50
    >       assert student.is_passed() is True
E      assert False is True
E      + where False = is_passed()
E      +   where is_passed = <test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.StudentResult object at 0x7b011577a960>.is_passed

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:109: AssertionError
_____ test_student_result_is_passed_full_marks _____

    def test_student_result_is_passed_full_marks():
        student = StudentResult("Ryan", 100)
    >       assert student.is_passed() is True
E      assert False is True
E      + where False = is_passed()
E      +   where is_passed = <test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.StudentResult object at 0x7b011577b8f0>.is_passed

test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py:121: AssertionError
===== short test summary info =====
FAILED test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_type_error_name_none
FAILED test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_type_error_name_non_string
FAILED test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_value_error_name_empty_string
FAILED test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_type_error_marks_none
FAILED test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_type_error_marks_non_numeric
FAILED test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_value_error_marks_too_low
FAILED test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_init_value_error_marks_too_high
FAILED test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_set_marks_type_error_name
FAILED test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_set_marks_type_error_non_numeric
FAILED test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_set_marks_value_error_too_low
FAILED test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_set_marks_value_error_too_high
FAILED test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_is_passed_above_threshold
FAILED test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_is_passed_at_threshold
FAILED test_student_result_generated_d7442b1089024ee58db2dd0c19cf9432.py::test_student_result_is_passed_full_marks
===== 14 failed, 6 passed in 0.64s =====
```

```
===== test session starts =====
platform linux -- Python 3.12.12, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /tmp/tmpv5d4kobo
plugins: langsmith-0.6.8, typeguard-4.4.4, anyio-4.12.1
collecting ... collected 20 items

test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_init_valid_data PASSED [ 5%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_init_valid_boundary_marks PASSED [ 10%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_init_type_error_name_none PASSED [ 15%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_init_type_error_name_non_string PASSED [ 20%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_init_value_error_name_empty_string PASSED [ 25%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_init_type_error_marks_none PASSED [ 30%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_init_type_error_marks_non_numeric PASSED [ 35%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_init_value_error_marks_too_low PASSED [ 40%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_init_value_error_marks_too_high PASSED [ 45%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_set_marks_valid PASSED [ 50%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_set_marks_valid_boundary PASSED [ 55%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_set_marks_type_error_none PASSED [ 60%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_set_marks_type_error_non_numeric PASSED [ 65%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_set_marks_value_error_too_low PASSED [ 70%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_set_marks_value_error_too_high PASSED [ 75%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_is_passed_above_threshold PASSED [ 80%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_is_passed_at_threshold PASSED [ 85%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_is_passed_below_threshold PASSED [ 90%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_is_passed_zero_marks PASSED [ 95%]
test_student_result_generated_830ceb274f2346b985b25e44599079e8.py::test_student_result_is_passed_full_marks PASSED [100%]

===== 20 passed in 0.22s =====
```

TASK_05:

Prompt: Generate pytest test cases for `is_valid_username(username)` validating length, spaces, and alphanumeric rules using TDD.

Code:

```
def is_valid_username(username):
    if len(username) < 5:
        return False
    if " " in username:
        return False
    if not username.isalnum():
        return False
    return True

print(is_valid_username("user01"))
print(is_valid_username("ai"))
print(is_valid_username("user name"))
print(is_valid_username("user@123"))
def is_valid_username(username):
    if len(username) < 5:
        return False
    if " " in username:
        return False
    if not username.isalnum():
        return False
    return True

print(is_valid_username("user01"))
print(is_valid_username("ai"))
print(is_valid_username("user name"))
print(is_valid_username("user@123"))
```

Output

```
... True
False
False
False
True
False
False
False
```

